

UNIVERSITEIT ANTWERPEN
Academiejaar 2025-2026

Faculteit Toegepaste Ingenieurswetenschappen

Computerarchitectuur

Labo computerarchitectuur: microcontrollers

Eric Paillet

Cursusdienst
UNIVERSITAS
Prinsesstraat 16
2000 Antwerpen
T +32 3 233 23 72
F +32 3 233 65 81
E info@cursusdienst.be
W www.universitas.be

**Bachelor of Science in de
industriële wetenschappen: elektronica-ICT**

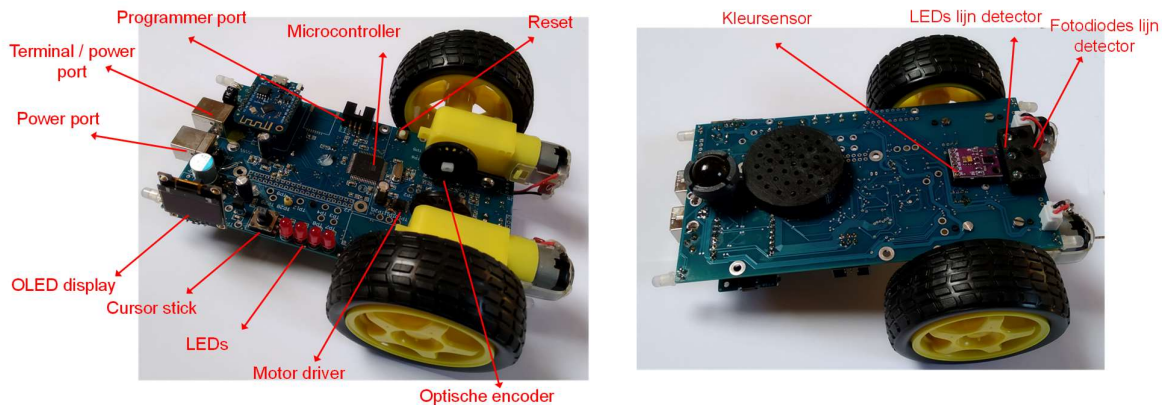
STUDIEGIDS NR 1533FTICPA

Inhoudstabel

1. Voorwoord & inleiding.....	3
2. Planning	4
3. Oefening 1: Atmel studio inleiding.....	5
4. Oefening 2: GPIO als output.....	9
5. Oefening 3: GPIO als input.....	11
6. Oefening 4: Timer counters	12
7. Oefening 5: GPIO interrupts toegepast op een incrementele encoder	13
8. Oefening 6: USART configuratie	15
9. Oefening 7: ADC	18
Appendix A Peripheral port mapping	20
Appendix B Linebot schema en PCB layouts	22
Appendix C FAQ	28
Appendix D Installatiehandleiding.....	30
Bibliografie.....	31

1. Voorwoord & inleiding

Deze cursusmodule maakt deel uit van het opleidingsonderdeel 4-Computerarchitectuur en Besturingssystemen en richt zich op het gebruik van microcontrollers. De nadruk ligt voornamelijk op de geïntegreerde periferiemodules van de microcontroller, zoals General Purpose IO (GPIO), communicatiepoorten, Analooq-naar-Digitaal Converters (ADC), Timers en Counters (TC). Tijdens deze cursus maak je gebruik van een eenvoudig robotplatform op wielen. Dit platform bevat een microcontroller en diverse actuatoren, sensoren en user interface-componenten. De gekozen componenten zijn specifiek geselecteerd om zo breed mogelijk de functionaliteit van de periferiemodules van de microcontroller te demonstreren.



In het practicum zal je drivers ontwikkelen voor verschillende componenten, zoals de LED's, cursorstick, motorsturing, motorencoder en optische lijnsensoren. Deze drivers dienen om de aansturing van de hardware te abstraheren. Vervolgens worden deze drivers gebruikt in het practicum Operating Systems om een complexere toepassing uit te werken.

De cursus omvat een reeks oefeningen die je zelfstandig kunt uitvoeren tijdens drie labosessies. De oefeningen zijn gebaseerd op de PowerPoint-presentatie en de XMEGA C Processor Manual. Vooral dit laatste document is van belang, omdat een van de doelstellingen van dit practicum is om vertrouwd te raken met Engelstalige technische documentatie.

In Appendix A vind je een overzicht van de pin-aansluitingen van de controller en hun mogelijke functies (pin mapping). Appendix B bevat de elektronische schema's van de PCB. Omdat je dicht op de hardware programmeert, is het essentieel om deze schema's bij de hand te hebben en goed te begrijpen. Appendix C bevat een FAQ (Frequently Asked Questions). Neem deze FAQ door, zodat je mogelijke problemen snel kunt herkennen tijdens het uitvoeren van de oefeningen. Appendix D bevat installatie-instructies voor de benodigde softwaretools, zodat je deze op je eigen pc kunt installeren.

Alle documentatie, software en PowerPoint-bestanden zijn beschikbaar op Blackboard onder: 4-Computerarchitectuur en Besturingssystemen → Opdrachten → Labo Microcontrollers: Periferie.

2. Planning

Hieronder vind je de planning voor de drie labozittingen terug. Vergeet niet voor elke oefening de aangeduide voorbereiding te maken. Indien je merkt dat je begint achter te lopen op de planning, neem dan zelf het initiatief om zittingen bij te werken buiten de geplande contacturen.

Zitting 1:

Theorie:

- Microcontrollers en XMEGA inleiding (PPT)
- GPIO module
- Timer counters

Uit te voeren:

- Oefening 1 (inleiding)
- Oefening 2 (GPIO output)
- Oefening 3 (GPIO input)

Zitting 2:

Theorie:

- USART module
- Interrupts

Uit te voeren:

- Oefening 4 (Timer/Counters)

Zitting 3:

Uit te voeren:

- Oefening 5 (GPIO interrupts)
- Oefening 6 (USART)
- Als uitbreiding: oefening 7 (ADC), theorie zelfstandig door te nemen.

3. Oefening 1: Atmel studio inleiding

Documentatie:

Zie PPT.

Voorbereiding: /

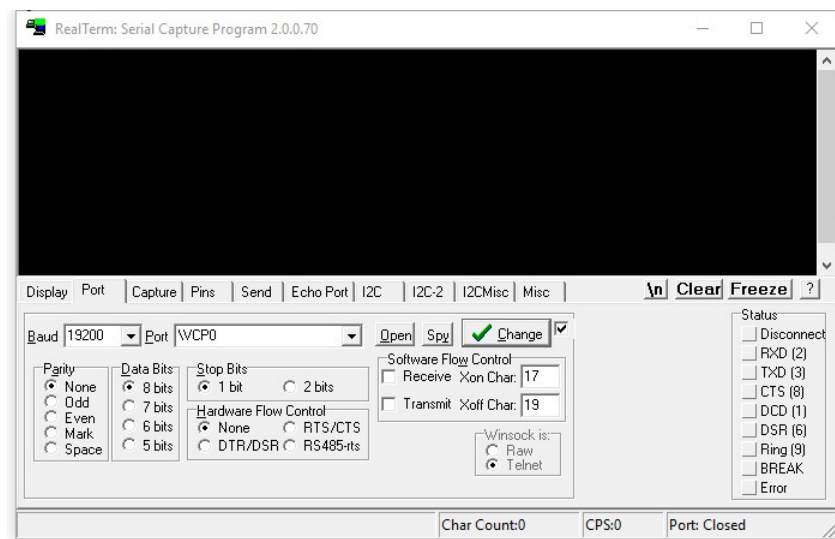
Beschrijving:

Deze oefening dient ter inleiding van het gebruik van Atmel studio in combinatie met de Atmel ICE / Atmel Dragon programmer en het robot platform. In deze oefening wordt het gebruik van de stap-voor-stap debugger ook overlopen.

Opdracht:

Koppel de programmer adapter aan je laptop. Koppel vervolgens de programmer adapter aan het robotwagentje via de 6-aderige flatcable connector aan de programmer port (zie p3). Verbind vervolgens je PC via een tweede USB kabel met de terminal / power port. Deze verbinding zal gebruikt worden om STDIO output (bv printf) op een terminal venster af te beelden en de robot te voeden.

Start RealTerm en klik op de 'Port' tab. Zet 'Port' op '\VCP0', de baudrate op 19200. Zorg dat 'hardware flow control' op 'None' staat. Klik vervolgens op 'Open'. Alle terminal output verschijnt nu in RealTerm.



Tekst verstuurd naar de terminal verschijnt in het zwarte venster. Je kan ook tekst naar de microcontroller sturen, door op het zwarte venster te klikken. Elke toetsdruk wordt doorgestuurd naar de microcontroller. Standaard zie je niet wat er doorgestuurd wordt. Om ook de verzonden tekens zichtbaar te maken, moet je de Display->Half duplex vink zetten.

Een alternatieve manier om ook binaire data te kunnen verzenden is via de 'Send' tab.

Download het 'LinebotTemplate1' project van blackboard. Pak dit startproject uit in een map naar keuze. Start vervolgens Atmel studio op. Deze ontwikkelingsomgeving is gebaseerd op Microsoft Visual Studio, en dient als IDE voor de opensource AVR-GCC compiler.

Open het startproject: *File->Open project/solution*. Open LinebotTemplate1.atsln

Dit project bevat reeds een algemene structuur en zal in de loop van de labozittingen aangevuld moeten worden. De projectstructuur ziet er als volgt uit:

```
Main.c
.....DriverADC.c
.....DriverADPS9960.c
.....DriverCursorstick.c
.....DriverLed.c
.....DriverMotor.c
.....DriverOled.c
.....DriverPower.c
.....DriverSysClk.c
.....DriverTWIMaster.c
.....DriverUSART.c
```

De 'main()' functie vinden we in 'main.c' terug.

De 'main()' functie is onderverdeeld in een aantal secties gemarkeerd door `####n###` commentaarregels. Deze secties hebben volgende taak:

`###1###`: Initialisatie van alle submodules. Elke submodule komt ruwweg met een hardware module overeen. Deze submodules zullen in de loop van de zittingen ingevuld worden.

`###2###` : Hier wordt een functie aangeroepen die een teller (0 tot 9) naar de terminal stuurt.

Laad de code in de microcontroller via *Debug->Start without debugging*.

Mogelijk verschijnt volgende foutmelding:



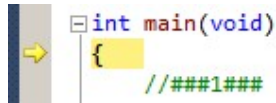
Indien dit het geval is, dient de programmer geselecteerd te worden: *Project->LinebotTemplate1 properties->tool->Selected debugger/programmer: Atmel ICE*.

Na een succesvolle upload zal de controller de code uitvoeren. Controleer of op de terminal (RealTerm) tekst komt ("counter:1"). Indien niets verschijnt, controleer de verbindingen en de seriële poort instellingen.

Opgelet: Zoals je merkt werkt de code niet naar behoren. De teller blijft hangen op "counter:1" terwijl deze eigenlijk van 0 tot 9 zou moeten lopen. We zullen nu proberen dit probleem te debuggen met hulp van de stap-voor-stap debugger.

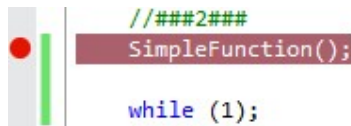
Stap-voor-stap mode werkt alleen betrouwbaar en voorspelbaar indien de optimalisatie instellingen van de compiler uitgeschakeld worden: *Project->LinebotTemplate1 properties->Toolchain->AVR/GNU C Compiler->Optimization->Optimization level:None (-O0)*. Het nadeel is dat de code qua snelheid en geheugenverbruik niet optimaal zal zijn. Indien de 'optimization level' niet op '0' gezet wordt, zullen sommige breakpoints mogelijk niet gezet kunnen worden doordat de compiler sommige stukken code herschikt / wist. Ook sommige variabelen kunnen mogelijk niet bekeken worden, gezien de compiler ongebruikte variabelen kan wissen / meerdere variabelen kan combineren op één geheugenlocatie.

Start het programma in stap-voor-stap mode door te klikken op *Debug->Start debugging and break (ALT-F5)*. Na upload stopt de debugger op de eerste regel van de 'main()' functie. De huidige locatie van de program counter wordt door een gele pijl weergegeven.



```
int main(void)
{
    //###1###
}
```

Zet nu een breekpunt aan het begin van de foutieve functie. Doe dit door in de marge (waar het rode bolletje in onderstaande afbeelding staat) te klikken.



```
//###2###
SimpleFunction();

while (1);
```

Wanneer het programma terug hervat, via *Debug->Continue (F5)* zal het uitvoeren tot aan het breekpunt. Doe dit nu.

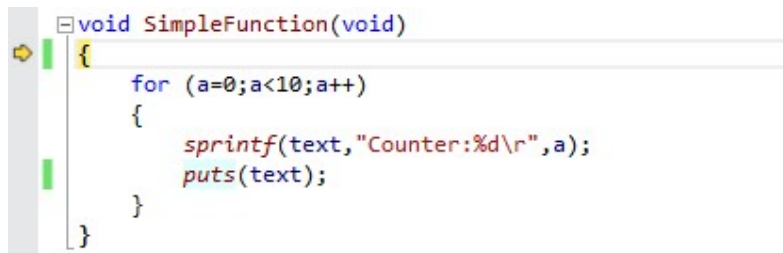


```
//###2###
SimpleFunction();

while (1);
```

We wensen nu verder stap voor stap door het programma te gaan en de functie 'SimpleFunction()' te onderzoeken. Dit kan via *Debug->Step into (F11)*. Indien je op *Debug->Step over (F10)* klikt, zal SimpleFunction() uitgevoerd worden en de processor pas onderbroken worden na de volledige uitvoering van deze functie.

We bevinden ons nu aan het begin van de functie:



```
void SimpleFunction(void)
{
    for (a=0;a<10;a++)
    {
        sprintf(text,"Counter:%d\r",a);
        puts(text);
    }
}
```


Vervolgens stappen we verder door de functie tot aan de puts() functie. Doe dit deze keer met behulp van *Step over (F10)* om te vermijden dat je in de code van de sprintf() of puts() functie duikt (hiervan is geen source code beschikbaar voor de debugger).

```

void SimpleFunction(void)
{
    for (a=0;a<10;a++)
    {
        sprintf(text,"Counter:%d\r",a);
        puts(text);
    }
}

```

We willen nu relevante variabelen ('text' en 'a') bekijken. Doe dit door in het watch window in de kolom 'Name' 'text' en 'a' in te vullen. Indien er geen watch window zichtbaar is, maak dit zichtbaar via *Debug->Window->Watch*. Je ziet nu de gewenste variabelen, hun inhoud en hun geheugenadressen (het @0x2024 achtervoegsel bij de variabele 'a' bijvoorbeeld).

Watch 1		
Name	Value	Type
text	0x201a	char[10]{data}@0x201a
[0]	67	char{data}@0x201a
[1]	111	char{data}@0x201b
[2]	117	char{data}@0x201c
[3]	110	char{data}@0x201d
[4]	116	char{data}@0x201e
[5]	101	char{data}@0x201f
[6]	114	char{data}@0x2020
[7]	58	char{data}@0x2021
[8]	48	char{data}@0x2022
[9]	13	char{data}@0x2023
a	0	char{data}@0x2024

Stap nog enkele malen door de lus, houd de variabelen in het oog. Je zult zien dat gewijzigde variabelen als extra hulpmiddel rood kleuren. Tracht nu aan de hand van het watch window een conclusie te formuleren waarom de code niet correct werkt. Corrigeer de fout en noteer de foutoorzaak in commentaar in je code. Zorg dat deze conclusie ook in je ingeleverde portfolio blijft staan.

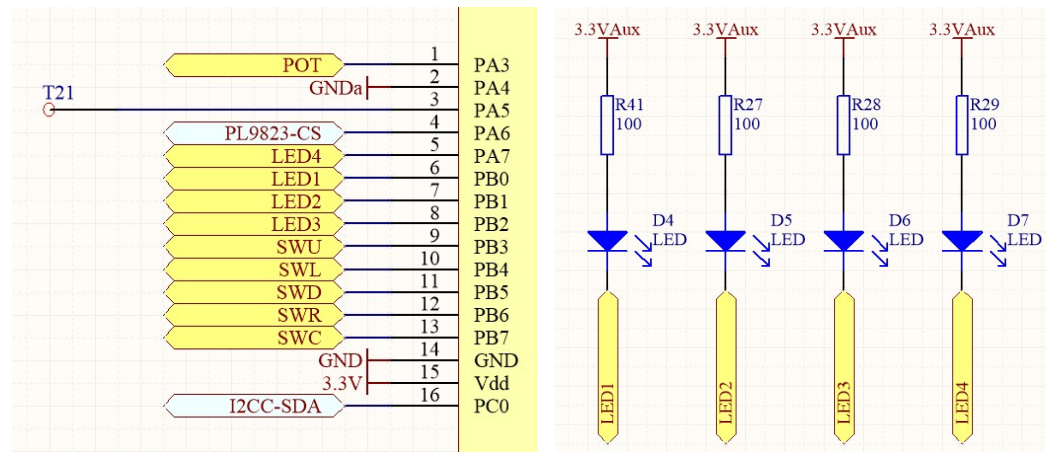
4. Oefening 2: GPIO als output

Documentatie:

Zie PPT, Processor manual p119 e.v. voor GPIO beschrijving, p129 e.v. voor GPIO registers

Beschrijving:

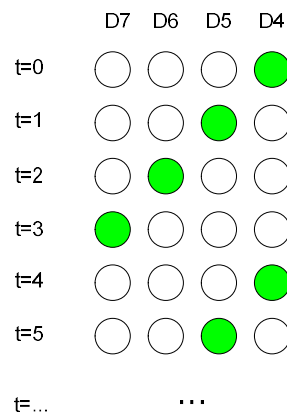
Op PB0, PB1, PB2 en PA7 zijn LEDs aangesloten.



Let op de wijze van aansluiting, bepaal het niveau (hoog / laag) op de GPIO pinnen om de LED's te doen branden.

Opdracht:

- Vervolledig de functie `DriverLedInit()` in `DriverLed.c`. Het doel is PB0 – PB2 en PA7 als output te schakelen. De poorten dienen zodanig geconfigureerd te worden dat bij het schrijven van een '1' naar een bit in het PORTB.OUT / PORTA.OUT register, de bijhorende LED brandt.
- Vervolledig de functie `DriverLedWrite()` in `DriverLed.c`. Deze functie aanvaardt een parameter 'LedData'. Bits 0 – 3 van deze parameter dienen respectievelijk gemapt te worden op PB0 – PB2 en PA7. Een '1' bit in 'LedData' betekent dat de overeenkomstige LED moet branden. Een '0' bit betekent dat de overeenkomstige LED gedoofd moet zijn. PB3 – PB7 en PA0 – PA6 mogen **NIET** gewijzigd worden. Deze lijnen moeten dus blijven staan in de toestand waarin ze stonden (zelfs als ze niet gebruikt worden) voor de functieoproep. Voer dit uit met behulp van de bitsgewijze operatoren in C.
- Maak in de hoofdloop van het programma (`main.c`) een looplicht. Elke cyclus van de hoofdloop dient het looplicht een stap te maken. Volgende sequentie moet dus uitgevoerd worden:



Uitbreiding:

- Vereenvoudig indien mogelijk je code (zo weinig mogelijk stappen).
- Schrijf de functies `DriverLedSet()`, `DriverLedClear()` en `DriverLedToggle()`. Deze functies dienen respectievelijk de LED's aan te zetten / doven / wisselen van toestand bij het schrijven van een '1' naar de overeenkomstige bitposities. Ook hier moeten niet gebruikte pinnen ongemoeid blijven.
- Schrijf een testroutine in `main.c` om bovenstaande functies uit te testen.

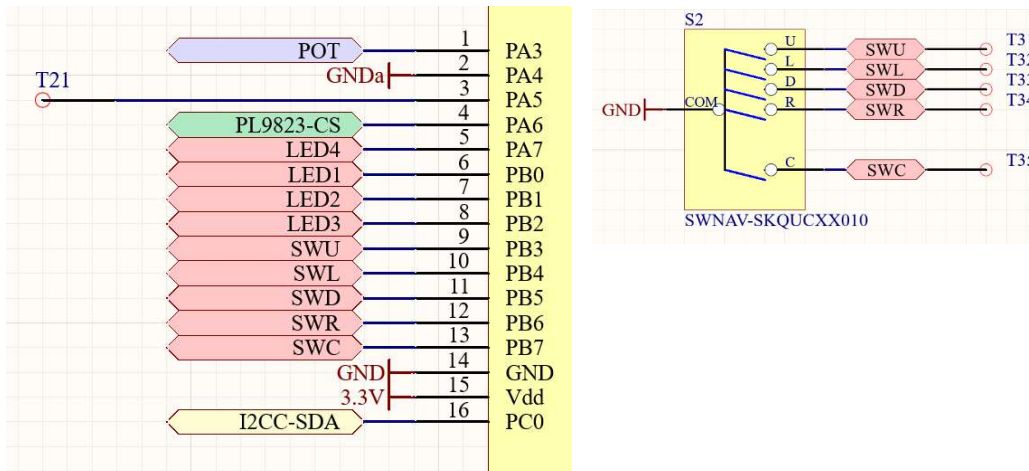
5. Oefening 3: GPIO als input

Documentatie:

Zie PPT, Processor manual p119 e.v. voor GPIO beschrijving, p129 e.v. voor GPIO registers

Beschrijving:

Op PB3 – PB7 is een cursor stick aangesloten. Dit is een set van 5 schakelaars die respectievelijk geactiveerd worden door de stick naar boven / links / onder / rechts te bewegen en in te drukken.



Opdracht:

- Vervolledig de functie DriverCursorstickInit() in DriverCursorstick.c. PB3 – PB7 dienen als input geconfigureerd te worden, zodanig dat de schakelaars uitgelezen kunnen worden. Een ingedrukte schakelaar moet overeen komen met een '1' in het PORTB.IN register.
- Vervolledig de functie DriverCursorstickGet (). Deze functie dient een waarde terug te geven die de toestand van de schakelaars weergeeft. De bits van deze waarde dienen als volgt gekoppeld te worden. **Let op de bitvolgorde!** Combinaties zijn mogelijk, bijvoorbeeld 'SWU' en 'SWL' kunnen gelijktijdig geactiveerd worden.

B7:	B6:	B5:	B4:	B3:	B2:	B1:	B0:
/	/	/	SWU	SWL	SWD	SWR	SWC

- Breid de hoofdloop van het programma (main.c) uit zodanig dat bij het bewegen van de cursorstick een melding op het terminal venster komt die de bewegingsrichting aangeeft.

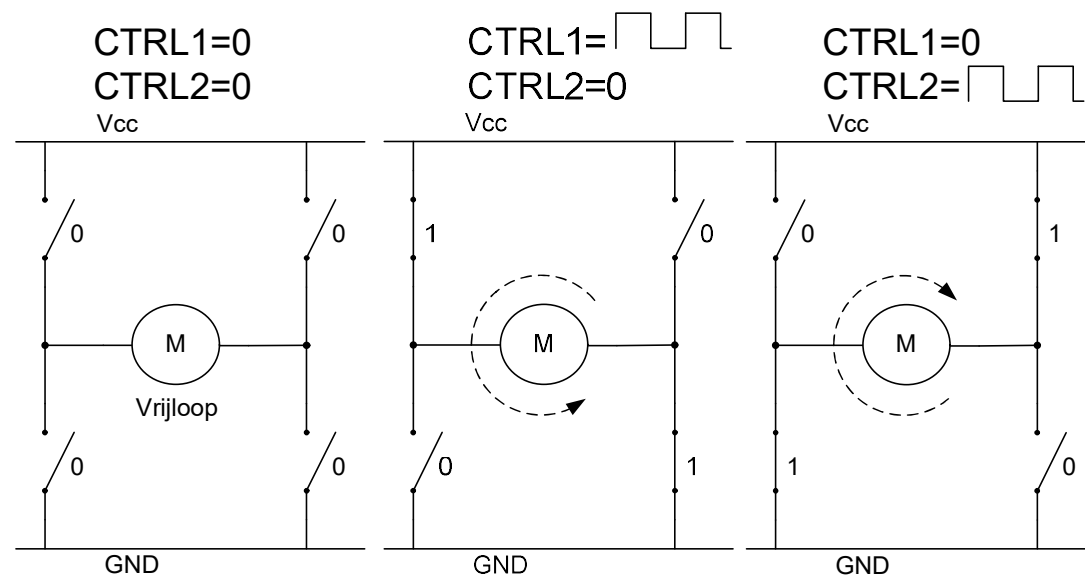
6. Oefening 4: Timer counters

Documentatie:

Zie PPT, Processor manual p142 e.v. voor TC beschrijving, p154 e.v. voor TC registers

Vorbereiding: Neem de bovenstaande documentatie door. Neem de opdracht door. Schrijf code om de opdracht uit te voeren, zodat je tijdens de labozitting de code alleen nog maar moet debuggen. De geschreven code vormt je voorbereiding.

Beschrijving: Op PF0-PF3 is een H-brug (DRV8833) aangesloten waarmee we de twee motoren van de robot kunnen aansturen. Onderstaande schema's geven aan hoe we de motoren in vrijloop kunnen laten of in de twee richtingen kunnen laten draaien. Door de duty cycle van een PWM signaal op CTRL1 of CTRL0 (afhankelijk van de gewenste draairichting) te variëren, kunnen we de motorsnelheid aanpassen.



Opdracht:

- Schrijf de functie `DriverMotorInit()` in `DriverMotor.c`. Configureer de H-brug controle lijnen als output. Vergeet ook de sleep pin van de H-brug niet! Configureer vervolgens TCF0 in 'single slope PWM' mode. Stel de prescaler en PER register zodanig in dat we een PWM frequentie van exact **10 kHz** bekommen. De CPU- en periferie klokfrequentie (f_{per}) is **32 MHz**.
- Schrijf de functie `DriverMotorSet()`. Deze functie aanvaardt twee snelheidsparameters, één voor elke motor. Deze parameter ligt in het interval **[-3000,+3000]**. Het teken van de parameter geeft de rotatierichting van de motor weer.
- Test de functies uit door een testroutine in `main.c` te schrijven. Deze routine dient via de terminal d.m.v. `scanf()` een waarde op te vragen aan de gebruiker. Deze waarde kan dan als snelheid gebruikt worden, welke doorgegeven wordt aan `DriverMotorSet()`.
- Meet het gegenereerde PWM signaal op d.m.v. een smartscope en neem de metingen mee op in je portfolio. Je kan gebruik maken van testpunten T22, T23, T24 en T30. Op de metingen moet duidelijk zijn dat de frequentie overeenkomt met wat hierboven gevraagd is. Toon ook aan dat de duty cycle correct is. Geef een waarde van **2250** door aan `DriverMotorSet()`. De gemeten duty cycle zou **75%** moeten zijn.

7. Oefening 5: GPIO interrupts toegepast op een incrementele encoder

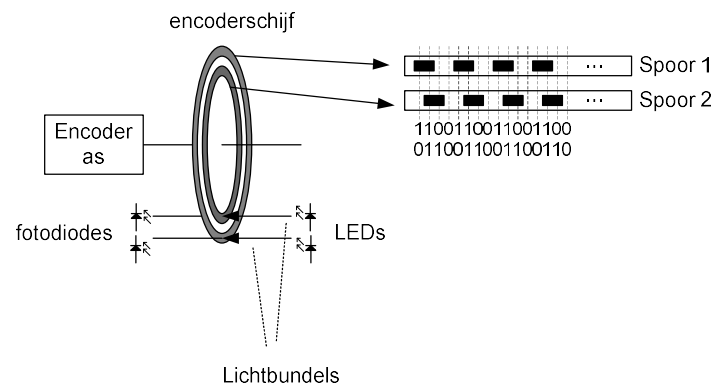
Documentatie:

Zie PPT, Processor manual p112 e.v. voor interrupt controller beschrijving, processor manual p125 e.v. voor GPIO interrupt beschrijving, 131 e.v. voor GPIO registers.

Vorbereiding: Neem de bovenstaande documentatie door. Neem de opdracht door. Schrijf code om de opdracht uit te voeren, zodat je tijdens de labozitting de code alleen nog maar moet debuggen. De geschreven code vormt je voorbereiding.

Beschrijving:

Op poort PC6, PC7, PE4 en PE5 zijn twee optische incrementele encoders aangesloten. Deze encoders worden gebruikt om de rotatiehoek van de wielen te meten. Deze informatie kan vervolgens in het practicum deel 'OS' gebruikt worden om de wielen aan een gewenste snelheid te laten draaien.



Hieronder staat een tabel die het werkingsprincipe demonstreert voor de eerste encoder, aangesloten op PC6 en PC7.

PC6	1	1	0	0	1	1	0	0	1	1	0	0
PC7	0	1	1	0	0	1	1	0	0	1	1	0
Gray code	1	2	3	0	1	2	3	0	1	2	3	0

De combinatie van de twee digitale signalen kunnen we schrijven als een gray code die optelt of aftelt, afhankelijk van de richting waarin de as gedraaid wordt. Op zich gaan we die gray code niet gebruiken. Wat we wel gaan doen is kijken onder welke voorwaarde de gray code optelt of aftelt. Dit wordt weergegeven in onderstaande tabel.

PC6	↑	↓	↑	↓	0	0	1	1
PC7	0	0	1	1	↑	↓	↑	↓
Stap	+	-	-	+	-	+	+	-

Afhankelijk van de combinatie stijgende / dalende flank van één lijn en de toestand van de andere lijn, maken we een stap in positieve of negatieve richting. Deze stappen worden geaccumuleerd in een globaal telregister. Op deze manier kunnen we de hoekpositie van de motoras sinds de opstart van het systeem bijhouden.

De flanken kunnen we detecteren aan de hand van twee GPIO interrupts, respectievelijk op de PC6 en PC7 pinnen.

Opdracht:

- Breid de functie `DriverMotorInit ()` functie in `DriverMotor.c` uit: Configureer de vier encoder lijnen om interrupts te geven op een willekeurige flank.
- Voer metingen op PC6 en PC7 uit met behulp van een oscilloscoop. Zorg dat je op beide lijnen pulsen meet als je aan de encoder as draait. Merk op hoe de flanken van de twee lijnen verschoven zijn t.o.v. elkaar. Merk ook op hoe de verschuiving omwisselt als je van rotatierichting wisselt. Verwerk de metingen in je portfolio.
- Schrijf de ISR's, stockeer de encoder positie in een globale variabele. LET OP: deze variabele moet het 'volatile' voorvoegsel hebben. Noteer in commentaar waarom. Zoek dit op, als je dit niet weet. OPGEPAST: vermijd het gebruik van `printf()` / `puts()` / andere stdio functies in de ISR's. De stdio functies zijn non-reentrant geïmplementeerd. Indien de oproep van deze functies in de hoofdlus onderbroken wordt door een tweede instantie in een ISR zal de processor vast blijven hangen. Als je toch stdio functies in de ISR gebruikt, vermijd dan dergelijke functies in de hoofdlus. Een tweede probleem is de lange uitvoeringstijd van de stdio functies. Hierdoor zullen snel opeenvolgende oproepen van de ISR verloren gaan.
- Schrijf de functie `DriverMotorGetEncoder()`. Deze functie dient de actuele encoderposities terug te geven onder de vorm van een voorgedefinieerde struct. De definitie van deze struct vind je terug in `DriverMotor.h`.
- Breid de functie `main()` in `main.c` uit om de encoder hoekposities af te printen op de terminal. Controleer de werking van je code door een wiel 360° in één richting te draaien, dan terug 360° in de andere richting. De weergegeven encoder positie zou terug op 0 moeten staan.

8. Oefening 6: USART configuratie

Documentatie:

Zie PPT, Processor manual p244 e.v. voor USART beschrijving, p258 e.v. voor USART registers

Voorbereiding: Neem de bovenstaande documentatie door. Neem de opdracht door. Reken de standaardinstelling van 19200 baud na. Bouw de berekeningstool op, zoals in de opgave gevraagd. De berekening en de tool vormen je voorbereiding.

Beschrijving:

USARTE wordt gebruikt om via een UART<->USB converter (FT232) een verbinding te maken met een terminal programma welke loopt op je PC. DriverUart.c bevat alle code nodig om te USART correct te configureren en om verzonden en ontvangen data af te handelen in 'polled' mode (dus zonder interrupts). Verder wordt in deze driver een verbinding gemaakt tussen de stdio functies putchar(), getchar(), puts(), gets(), printf(), scanf() enz... en de USART driver.

Hieronder wordt de driver code voor de USART overlopen:

```
void DriverUSARTInit(void)
{
    USART_PORT.DIRSET=0b00001000;
    USART_PORT.DIRCLR=0b00000100;
```

De DriverUSARTInit() functie wordt aangeroepen in het begin van het programma om de USART te initialiseren. In bovenstaande code wordt de transmit lijn als output geschakeld, de receive lijn als input. 'USART_PORT' is een macro die in hwconfig.h geïnitieerd is als PORTE (voor USARTE).

```
    USART.CTRLA=0b00000000;
```

USART is in hwconfig.h gedefinieerd als USARTE. USARTx.CTRLA stelt de interrupt prioriteit van de USART interrupt bronnen in (zie proc. manual). De USART wordt in 'polled' mode gebruikt. Interrupts worden dus uitgeschakeld.

```
    USART.CTRLB=0b00011000;
```

De USART transmitter en receiver worden beide aangeschakeld. Double speed communicatie mode wordt uitgeschakeld (16 USART klokcycli per bit).

```
    USART.CTRLC=0b00000011;
```

De USART wordt in asynchrone mode (UART) geschakeld met volgende eigenschappen: geen pariteitsbit, 1 stopbit, 8 databits.

```
    USART.BAUDCTRLA=0xE5; //BSEL=3301, BSCALE=-5 19200 baud
    USART.BAUDCTRLB=0xBC;
```


Deze registers stellen de baudrate in. Informatie over de baudrate generator kan gevonden worden in de processor manual, p245 e.v. . Er zijn meerdere combinaties van BSEL en BSCALE mogelijk om een bepaalde baudrate te bekomen. De USART staat ingesteld op 19200 baud. Reken dit na. Hou rekening met het feit dat de CPU klok (f_{per}) ingesteld staat op 32 MHz. Door het feit dat zowel BSEL als BSCALE gehele getallen zijn, zal er altijd een verschil zijn tussen de berekende (afgeronde) baudrate en de gewenste baudrate. Door een geschikte combinatie van BSEL en BSCALE te nemen kan dit verschil zo klein mogelijk gehouden worden.

```
static FILE UsartStdio = FDEV_SETUP_STREAM(stdio_putchar, stdio_getchar,
_FDEV_SETUP_RW);
```

...

```
    stdout=&UsartStdio;
    stdin=&UsartStdio;
}
```

Bovenstaande code vormt een koppeling tussen de drivercode en de stdio library. We kunnen zelf functies (stdio_putchar en stdio_getchar) definiëren die de onderste laag (byte schrijven, byte lezen) vormen van de stdio library. Functies zoals putchar(), getchar(), printf(), scanf(),... steunen hierop. Meer informatie vind je in de LIBC user manual, p190 e.v.

```
static int stdio_putchar(char c, FILE * stream)
{
    USART.DATA = c;
    while (!(USART.STATUS & 0b01000000));
    USART.STATUS=0b01000000;
    return 0;
}
```

Deze functie vormt de basis van de 'putchar()' functie, waarmee één byte over de USART verstuurd wordt. De data wordt effectief weggeschreven door te schrijven in het USART.DATA register. De functie wacht tot de TXCIF (Transmit Complete interrupt flag) in het USART.STATUS register gezet is. Deze vlag wordt gezet wanneer de databyte verzonden is. Om voor te bereiden op de volgende datatransfer moet de vlag manueel op 0 gezet worden door een 1 (!!!) te schrijven naar de overeenkomstige bitpositie.

```
static int stdio_getchar(FILE *stream)
{
    while (!(USART.STATUS & 0b10000000));
    return USART.DATA;
}
```

De getchar() functie wacht tot er een byte klaar staat in de USART receive buffer, geeft deze terug. Er staat een byte in de ontvangst buffer klaar wanneer de RXCIF (Receive complete interrupt flag) bit in het USART.STATUS register gezet is.

Opdracht:

- Herconfigureer de USART om aan 115200 baud te werken i.p.v. de standaard ingestelde 19200 baud. Om dit te doen zal je een functie moeten schrijven welke de optimale BSCALE en BSEL berekent om een willekeurig instelbare baud rate te bekomen. Zorg dat deze functie ook de afwijking (absoluut en relatief) teruggeeft. Schrijf vervolgens de bekomen BSCALE en BSEL op een correcte manier automatisch weg in het BAUDCTRLA en BAUDCTRLB register. Tip: gezien bovenstaande functionaliteit vrij complex is, kan het handig zijn de functies lokaal op je laptop te programmeren en te testen vooraleer je ze integreert in het microcontroller project.
- Test via Realterm of de USART correct werkt op de nieuwe baudrate.
- Zend de byte '0x55' over de USART. Waarom we '0x55' zenden, wordt duidelijk wanneer je deze byte in binaire notatie bekijkt. Registreer met een oscilloscoop het resultaat op de transmit lijn van USARTE (T6). Sla het oscilloscoop beeld op en noteer op het beeld de startbit, databits, stopbit. Stel de cursors in op de oscilloscoop om een bitperiode te meten. Zorg dat deze bitperiode ook zichtbaar is op het oscilloscoop beeld. Sla het bewerkte beeld op in je projectfolder onder de naam USARTMeting1.jpg.

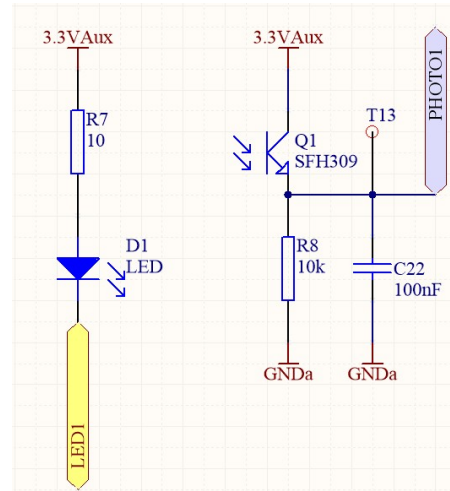
9. Oefening 7: ADC

Documentatie:

Zie PPT, Processor manual p279 e.v. voor ADC beschrijving, p290 e.v. voor ADC registers

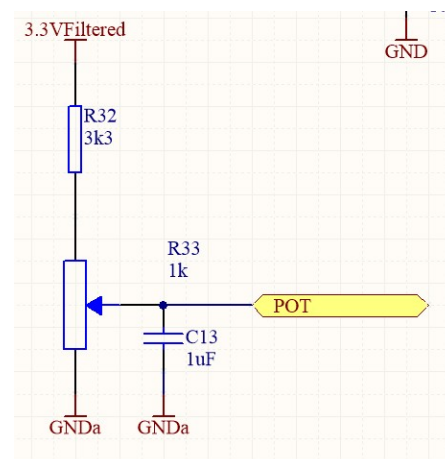
Voorbereiding: Neem de bovenstaande documentatie door. Neem de opdracht door. Schrijf code om de opdracht uit te voeren, zodat je tijdens de labozitting de code alleen nog maar moet debuggen. De geschreven code vormt je voorbereiding.

Beschrijving: Op het robotplatform zijn drie optische lijnvolger sensoren bevestigd. Deze sensoren zijn gebaseerd op een LED en een fototransistor. Ze vormen op deze manier een reflectiesensor die naar de grond is gericht. De fototransistor levert in combinatie met zijn serieweerstand (bv R8) een spanning tussen 0V en 1V afhankelijk van de reflectiegraad van het onderliggend oppervlak. Deze variabele spanning kan met behulp van de ingebouwde Analooog --> Digitaal Converter (ADC) van de microcontroller omgezet worden in een digitale waarde.



Verder vind je een potentiometerschakeling terug welke ook aan een ADC kanaal is gekoppeld. De potentiometer is in eerste instantie bedoeld om een variabele analoge spanning te kunnen aanleggen aan een ADC kanaal om deze gemakkelijk te kunnen uittesten.

Zowel voor de potentiometer als voor de lichtsensor kanalen wordt gemeten t.o.v. het GNDa net. Dit is een speciaal grond net, afgezonderd van het digitale grond net. Hiermee vermijden bij de metingen de ruis inherent aan het digitale circuit. De bedoeling is in differentiële mode te meten t.o.v. dit analoge grond net. GNDa is rechtstreeks gekoppeld aan de ADC4 (PA4) pin.



Opdracht:

- We beginnen eerst met het schrijven van een generieke driver die het mogelijk maakt via de ADC spanningsmetingen uit te voeren. Dit zowel in differential mode als single ended mode. In onze toepassing zullen we niet alle mogelijkheden van de ADC driver gebruiken, gezien de lijnsensoren en potentiometer enkel in differential mode worden gebruikt. Als oefening dient de ADC driver wel volledig uitgevoerd te worden.
- Vervolledig de functie DriverAdcInit() in DriverAdc.c. De ADC dient geïnitieerd te worden, met als referentiebron de interne spanningsreferentie van 1V. In DriverAdcInit () dienen alle gemeenschappelijke ADC instellingen gezet te worden, onafhankelijk van de pinnen waarop we meten en in welke mode (single ended of differential).
- Schrijf de functie DriverAdcGetCh(). Parameters en return waarde vind je terug in DriverAdc.h. Deze functie dient één meting uit te voeren op het via de parameters ingestelde kanaal, deze terug te geven. Je hoeft voor deze functie geen gebruik te maken van interrupts.

Volgende acties moeten opeenvolgend uitgevoerd worden in deze functie:

- Configuratie van de ADC module in functie van de parameters. Gebruik 12 bit left adjusted mode. Ga na wat 'left adjusted' betekent.
- Opstart van de meting (niet in freerun mode werken!)
- Wachten op meetresultaat
- Resultaat ophalen
- Breid de main() functie in main.c uit zodanig dat elke cyclus een meting uitgevoerd wordt op de drie lijnsensoren en potentiometer. Het resultaat van de metingen moet op de terminal naar buiten gebracht worden.
- Controleer of de ADC voor het potentiometer kanaal een bereik geeft van 0 tot 25000. Controleer ook of lichtveranderingen invloed hebben op de metingen van de lichtsensor kanalen.

Appendix A Peripheral port mapping

(Ref: XMEGA C3 datasheet p 53 e.v.)

PORT A	PIN #	INTERRUPT	ADCA POS/ GAIN POS	ADCA NEG	ADCA GAINNEG	ACA POS	ACA NEG	ACA OUT	REFA
GND	60								
AVCC	61								
PA0	62	SYNC	ADC0	ADC0		AC0	AC0		AREFA
PA1	63	SYNC	ADC1	ADC1		AC1	AC1		
PA2	64	SYNC/ASYNC	ADC2	ADC2		AC2			
PA3	1	SYNC	ADC3	ADC3		AC3	AC3		
PA4	2	SYNC	ADC4		ADC4	AC4			
PA5	3	SYNC	ADC5		ADC5	AC5	AC5		
PA6	4	SYNC	ADC6		ADC6	AC6		AC1OUT	
PA7	5	SYNC	ADC7		ADC7		AC7	AC0OUT	

PORT B	PIN #	INTERRUPT	ADCA POS	REFB
PB0	6	SYNC	ADC8	AREFB
PB1	6	SYNC	ADC9	
PB2	8	SYNC/ ASYNC	ADC10	
PB3	9	SYNC	ADC11	
PB4	10	SYNC	ADC12	
PB5	11	SYNC	ADC13	
PB6	12	SYNC	ADC14	
PB7	13	SYNC	ADC15	
GND	14			
VCC	15			

PORT C	PIN #	INTERRUPT	TCC0 ⁽¹⁾⁽²⁾	AWEXC	TCC1	USARTC0 ⁽³⁾	SPIC ⁽⁴⁾	TWIC	CLOCKOUT ⁽⁵⁾	EVENTOUT ⁽⁶⁾
PC0	16	SYNC	OC0A	$\overline{OC0ALS}$				SDA		
PC1	17	SYNC	OC0B	OC0AHS		XCK0		SCL		
PC2	18	SYNC/ ASYNC	OC0C	$\overline{OC0BLS}$		RXD0				
PC3	19	SYNC	OC0D	OC0BHS		TXD0				
PC4	20	SYNC		$\overline{OC0CLS}$	OC1A		$\overline{SS}$			
PC5	21	SYNC		OC0CHS	OC1B		MOSI			
PC6	22	SYNC		$\overline{OC0DLS}$			MISO		RTCOUT	
PC7	23	SYNC		OC0DHS			SCK		clk _{PER}	EVOUT
GND	24									
VCC	25									

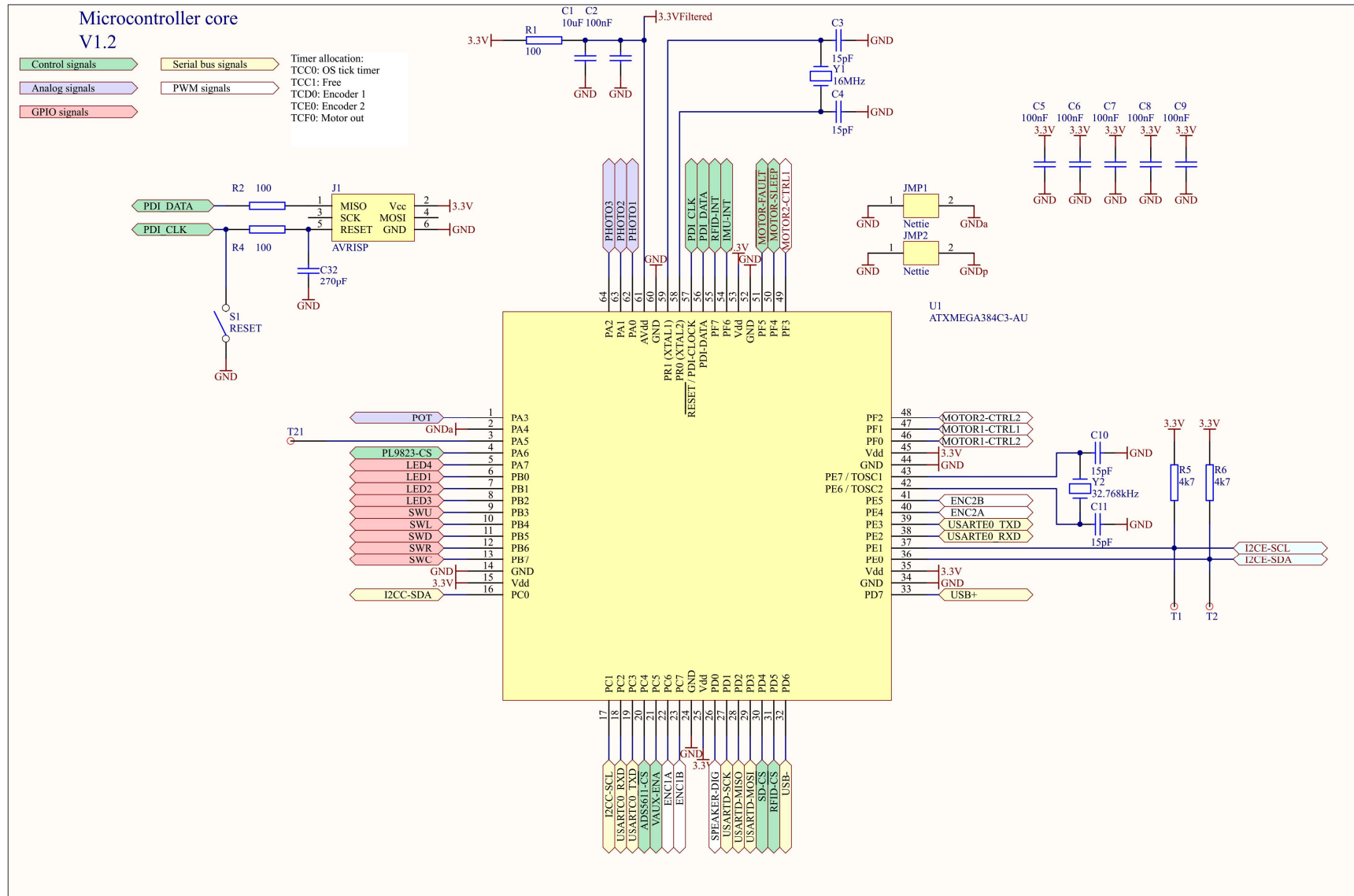
PORT D	PIN #	INTERRUPT	TCD0	USARTD0	SPID	USB	CLOCKOUT	EVENTOUT
PD0	26	SYNC	OC0A					
PD1	27	SYNC	OC0B	XCK0				
PD2	28	SYNC/ ASYNC	OC0C	RXD0				
PD3	29	SYNC	OC0D	TXD0				
PD4	30	SYNC			$\overline{SS}$			
PD5	31	SYNC			MOSI			
PD6	32	SYNC			MISO	D-		
PD7	33	SYNC			SCK	D+	Clk _{PER}	EVOUT
GND	34							
VCC	35							

PORT E	PIN #	INTERRUPT	TCE0	USARTE0	TOSC	TWIE	CLOCKOUT	EVENTOUT
PE0	36	SYNC	OC0A			SDA		
PE1	37	SYNC	OC0B	XCK0		SCL		
PE2	38	SYNC/ ASYNC	OC0C	RXD0				
PE3	39	SYNC	OC0D	TXD0				
PE4	40	SYNC						
PE5	41	SYNC						
PE6	42	SYNC			TOSC2			
PE7	43	SYNC			TOSC1		Clk _{PER}	EVOUT
GND	44							
VCC	45							

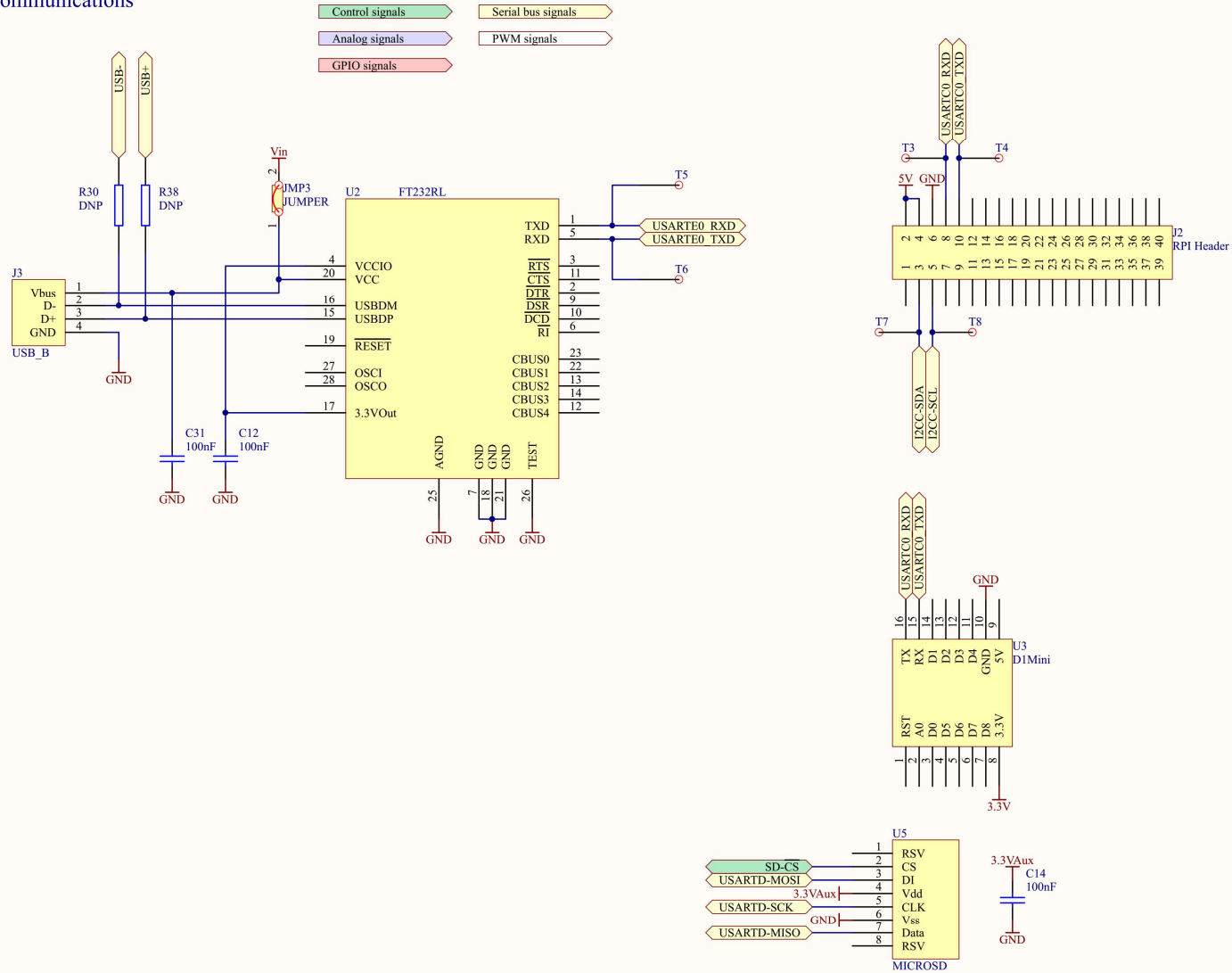
PORT F	PIN #	INTERRUPT	TCF0
PF0	46	SYNC	OC0A
PF1	47	SYNC	OC0B
PF2	48	SYNC/ ASYNC	OC0C
PF3	49	SYNC	OC0D
PF4	50	SYNC	
PF5	51	SYNC	
PF6	54	SYNC	
PF7	55	SYNC	
GND	52		
VCC	53		

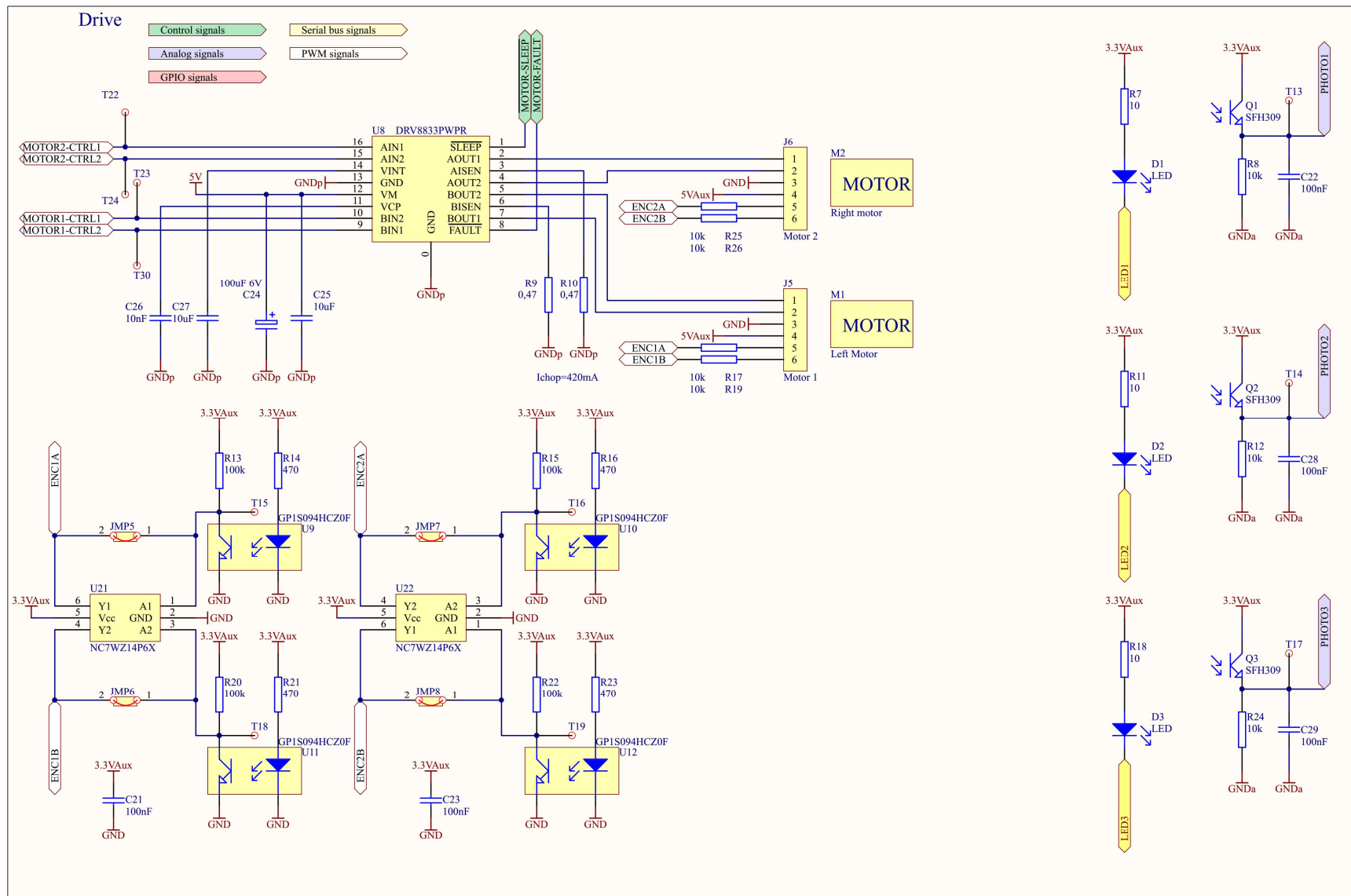
PORT R	PIN #	INTERRUPT	PDI	XTAL
PDI	56		PDI_DATA	
$\overline{RESET}$	57		PDI_CLOCK	
PRO	58	SYNC		XTAL2
PR1	59	SYNC		XTAL1

Appendix B Linebot schema en PCB layouts

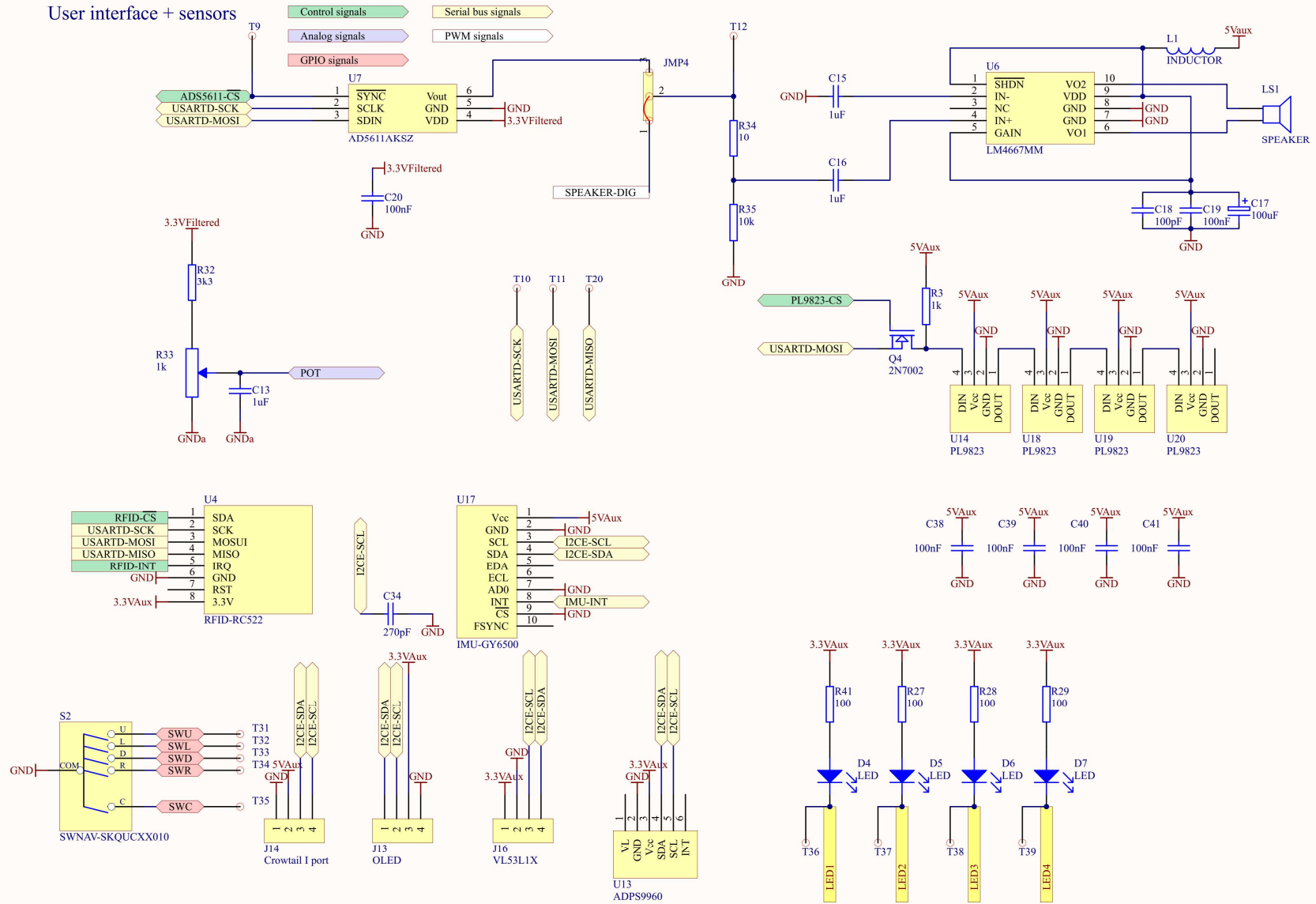


Communications

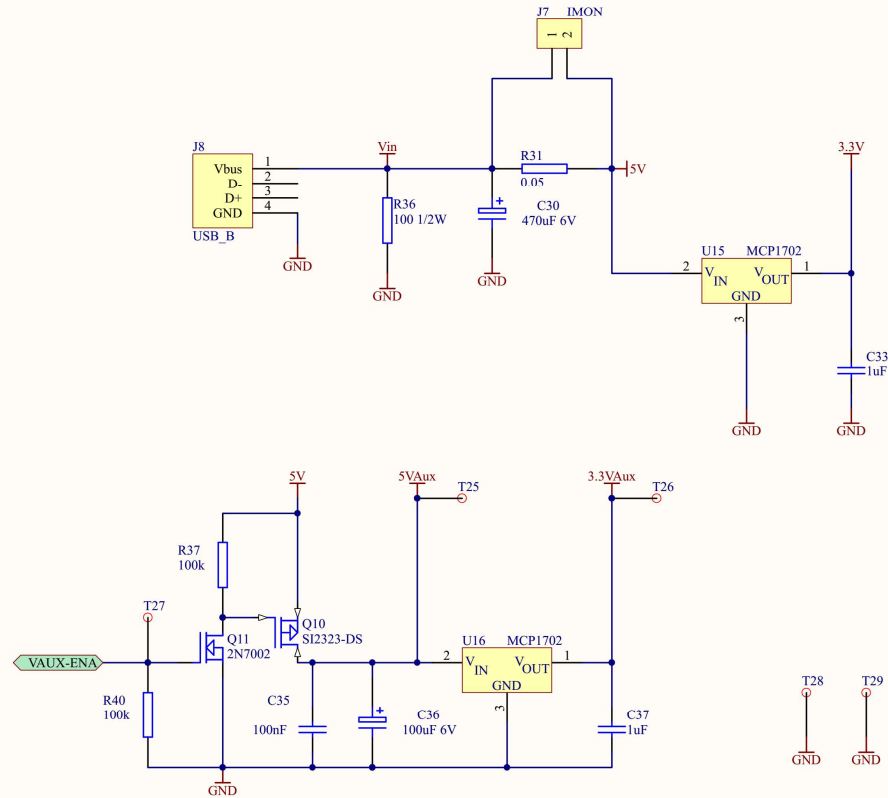
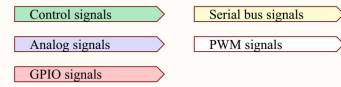


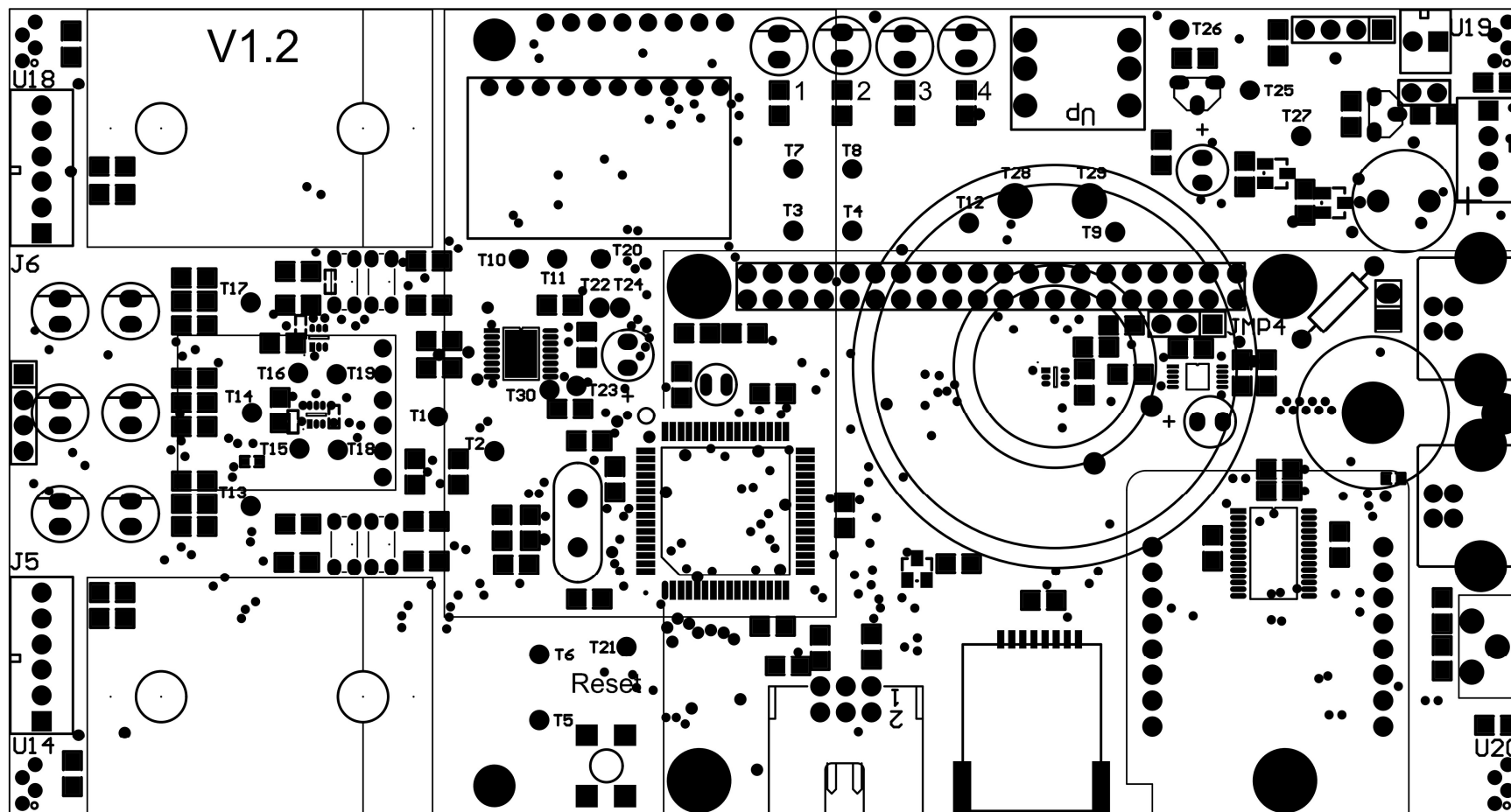


User interface + sensors



Power supply





Appendix C FAQ

ATMEL-ICE / DRAGON programmer wordt niet herkend/gezien bij een programmeerpoging.

Probeer volgende stappen. Indien een stap niet helpt, probeer de volgende.

- Herstart Atmel Studio.
- Trek de USB stekker van de programmer uit, steek deze terug in.
- Herinstalleer de programmer driver. Gebruik de tool 'InstallAtmelUSB' onder 'C:\Program Files\Atmel\AtmelUSBInstaller\JungoDriver\usb32' of 'C:\Program Files\Atmel\AtmelUSBInstaller\JungoDriver\usb64'.

Foutmelding tijdens programmeren / debuggen: 'Cannot connect TCF to USB...'

Herstart Atmel studio.

Tijdens het stap voor stap debuggen zou een gele pijl moeten komen op de uit te voeren regel. Dit gebeurt niet.

Doe 'Build->Clean solution' en daarna 'Build->Rebuild solution'.

De functies _delay_ms() en _delay_us() geven een onjuiste vertraging.

Volgende voorwaarden moeten voldaan zijn:

- Compiler optimization moet op 'O2' staan. Zie 'Project->Gamecontroller properties->Toolchain->AVR/GNU C Compiler->Optimization
- Include "hwconfig.h". Volgende macro's zijn van belang:

```
#define F_CPU 32000000L
#include <util/delay.h>
```

Een ISR wordt niet uitgevoerd, terwijl dit wel verwacht wordt.

- In de C file waarin de ISR voorkomt moet het bestand 'avr/interrupt.h' in #include staan. Indien dit de oorzaak is, verschijnt tijdens het compileren een warning betreffende de interrupt vector.

Bij het inpluggen van de programmer + robot PCB reageert de muispointer niet meer / verspringt willekeurig op het scherm.

- Bij het inpluggen van de programmer begint onder bepaalde omstandigheden (afhankelijk van de geprogrammeerde code) de robot onmiddellijk data over de USB seriële poort te sturen. Deze data kan misgeïnterpreteerd worden door windows als een seriële muis die aangesloten wordt. Oplossing: eerst de programmer connecteren, enige seconden later pas de robot PCB connecteren aan de programmer.

Bij het compileren van de code krijg je een melding ‘make access denied’

- Voer Atmel studio uit als Administrator.

printf van een floating point getal werkt niet

- Standaard is een vereenvoudigde versie van printf gelinkt, dit om geheugen uit te sparen. Om de volledige versie te linken moet je volgende stappen uitvoeren:

Project→properties→AVR/GNU linker→general: vink ‘use vprintf library’ aan.

Project→properties→AVR/GNU linker→miscellaneous: voeg -lprintf_flt toe.

Het lukt niet om via realterm / putty data door te sturen naar de microcontroller. Er komen geen karakters op de seriële lijn.

Controleer of onder de ‘port’ tab, hardware flow control op ‘none’ staat. Indien niet, pas aan.

Het lukt niet om via realterm / putty data door te sturen naar de microcontroller. Er komen rare tekens op mijn terminal venster.

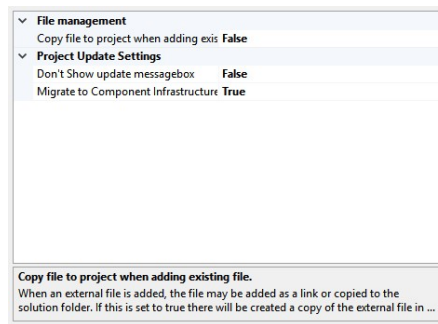
Controleer of de baudrate in realterm overeenkomt met de baudrate van de microcontroller firmware.

Appendix D Installatiehandleiding

Dit hoofdstuk biedt een korte handleiding om manueel de benodigde tools te installeren op je eigen PC.

Installatie van ATMEL Studio

- Download de ATMEL Studio installatiebestanden vanaf blackboard (4-computerarchitectuur→opdrachten→microcontrollers:periferie), pak de bestanden uit.
- Voer de installer uit.
- Start Atmel Studio
- Zet de instelling 'Copy file to project when adding existing file' op False via Tools-->Options-->Projects-->Miscellaneous



Installatie van Realterm

- Zie de link op blackboard

Installatie van student startproject

- Download het student startproject zip bestand (LinebotTemplate1.zip)
- Pak het bestand uit in een folder naar keuze

Bibliografie

1. Atmel ATXMEGA384C3 Datasheet:
http://ww1.microchip.com/downloads/en/devicedoc/atmel-8361-8-and-16-bit-avr-xmega-microcontrollers-atxmega384c3_datasheet.pdf
2. XMEGA C processor manual:
http://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-8465-8-and-16-bit-AVR-Microcontrollers-XMEGA-C_Manual.pdf
3. AVR LIBC reference: <http://nongnu.org/avr-libc/user-manual/index.html>